DESIGN PATTERN MAPPING
======================

1. STRATEGY
   Applied to : PricingStrategy (regular, flash, coupon), delivery selection
   Solves   : Swap pricing/delivery rules without editing core code
   Trade-off : More classes; selection logic needed

2. FACTORY
   Applied to : PaymentProviderFactory (by method/region/config)
   Solves   : Central creation of adapters, decorated with breaker/retry
   Trade-off : Registry must be kept in sync with config

3. STATE
   Applied to : ReservationState, OrderState
   Solves   : Legal transitions are explicit; illegal ones are rejected
   Trade-off : Many small classes; DB transition table must match

4. OBSERVER / PUB-SUB
   Applied to : Domain events via Kafka -> Shipment, Notification, Inventory
   Solves   : Loose coupling; new consumers without touching producers
   Trade-off : Eventual consistency; harder tracing

5. ADAPTER
   Applied to : RazorpayAdapter, StripeAdapter, courier adapters
   Solves   : Isolates vendor APIs behind our interface
   Trade-off : Lowest-common-denominator feature set

6. REPOSITORY
   Applied to : InventoryRepository, PaymentRepository, OrderRepository
   Solves   : Hides SQL/Redis; testable; transaction boundary in one place
   Trade-off : Extra layer; leaky for complex queries

7. FACADE
   Applied to : ReservationFacade, CheckoutFacade
   Solves   : One simple entry point orchestrating guard -> gate -> DB ->
event
   Trade-off : Can grow into a god object if not kept thin

8. CIRCUIT BREAKER (+ Retry, Bulkhead)
   Applied to : ResilientProvider around gateway/courier
   Solves   : Stops cascading failures and thread exhaustion on a slow gateway
   Trade-off : Needs tuning; may reject healthy calls briefly

9. OUTBOX
   Applied to : Every service that writes to DB and publishes an event
   Solves   : Atomic state change + event (no dual-write loss)
   Trade-off : Relay latency; duplicates, so consumers must be idempotent

10. DECORATOR
   Applied to : ResilientProvider wraps any PaymentProvider
   Solves   : Adds resilience without changing adapters
   Trade-off : Stack order matters

EXTENDING WITH MINIMAL CHANGE

- New payment provider : implement PaymentProvider, register in the factory.